

Feasibility of a Human-Built Generational Robot Software Factory

Executive summary

Yes, LEGO-based kits can take custom instructions and custom programs, but the practical answer is ecosystem-specific. LEGO Education SPIKE Prime supports block coding and Python in the SPIKE app; the SPIKE Prime hub is a programmable controller with six LPF2 ports, an IMU, speaker, LED matrix, and embedded MicroPython. LEGO MINDSTORMS Robot Inventor supported Scratch-style blocks and Python, and although the product retired in 2023, LEGO says its hub can still be programmed through the SPIKE app. LEGO Powered Up supports app-based block programming and also has an official BLE wireless protocol document for advanced integrations. LEGO EV3, though retired, still has support paths through EV3 MicroPython with Visual Studio Code and MakeCode. In short: **yes, LEGO can be programmed**, but the best long-term platform for a new “learning robot that designs its successors” is usually **SPIKE Prime or Powered Up plus external compute**, not a legacy MINDSTORMS-only workflow. ¹

The larger concept the question asks about is feasible **in a bounded, grammar-constrained form** today. A software-factory pipeline can maintain a parts inventory, generate robot morphologies from a restricted library of compatible building blocks, simulate and score them, compile build instructions and bills of materials, let a human assemble the selected design, run commissioning tests, and feed observed performance back into the next design cycle. That model is well aligned with both classic automated robot design work and newer morphology-and-control co-optimization work, including the early Nature paper by Lipson and Pollack, recent fast de novo robot design work published in PNAS, and more recent work that explicitly adds manufacturability constraints to generated morphologies. ²

The strongest near-term conclusion is this: **the “learning robot designs the next generation; a human builds it” model is realistic if the design space is constrained by a modular kit and an explicit assembly grammar**. It is **not** realistic, on today’s consumer educational platforms, to expect open-ended self-reproduction, autonomous fabrication of new connectors, or reliable invention of arbitrary new mechanisms without external CAD, simulation, and human review. The finite port counts, proprietary connectors, fixed actuator classes, and practical assembly tolerances of LEGO, VEX, Cubelets, and even many open-source kits make the design problem manageable only when it is constrained. That limitation is not a weakness; it is what makes the concept implementable. ³

For a first prototype, the best choices split by goal. If the goal is **education-first, fast iteration, and low mechanical risk**, use **SPIKE Prime plus Raspberry Pi Build HAT** or **SPIKE Prime plus an external laptop/webcam**. If the goal is **mechanically repeatable evolution with better sensing and stronger structures**, use **VEX V5**. If the goal is **research-grade openness and simulation-to-real continuity**, use **ROBOTIS OpenMANIPULATOR-X and/or TurtleBot3/TurtleBot4**. If the goal is **maximum modular actuation fidelity and research flexibility**, HEBI is technically excellent but often cost-prohibitive for early exploration. ⁴

What is technically feasible now

A workable “robot software factory” for successive generations has five ingredients. First, a **typed parts grammar** that knows which beams, plates, brackets, hubs, sensors, and actuators can connect. Second, a **robot description layer** for geometry, kinematics, mass, and sensors. Third, a **simulation and training layer** for evaluating designs and controllers before a human builds anything. Fourth, an **instruction compiler** that turns a selected design into build steps, port maps, and tests. Fifth, a **commissioning loop** that measures what the built robot actually does and learns from the gap between simulated and real performance. Those ingredients map well to existing standards and tools: URDF and SDF for robot descriptions, BrickLink Studio and LDraw-style files for LEGO geometry and instructions, HEBI’s YAML robot-config and HRDF stack for modular actuators, and Gazebo plus ROS 2 for simulation and system integration. ⁵

The fundamental feasibility constraint is that **morphology search must happen over a restricted design language**, not unrestricted free-form CAD. The historical literature is clear on this point. Lipson and Pollack evolved robots from basic building blocks; Matthews and colleagues optimized voxel-like soft robot designs rapidly from structured parameterizations; DERL evolved morphology and control jointly under explicit environmental tasks; Text2Robot added geometric processing and manufacturability-aware body-control co-optimization. None of these systems assume unconstrained physical fabrication from arbitrary parts bins. A LEGO-, VEX-, or DYNAMIXEL-based factory should therefore search over typed assemblies such as wheeled bases, arm segments, sensor mast variants, gripper choices, gear trains, and body layouts, not over arbitrary solids. ⁶

That bounded version is especially feasible because the candidate platforms already expose the right hooks. SPIKE Prime’s hub is programmable, has six ports, and supports third-party expansion concepts through its official “open platform for advanced users and third parties” language and the Maker Plate / Raspberry Pi Build HAT pathway. VEX V5 provides typed device APIs, auto-detected smart ports, stored programs, desktop and browser workflows, metal structural parts, and an AI Vision Sensor with AprilTag support. ROBOTIS provides ROS-compatible, open-source hardware and software in OpenMANIPULATOR-X and TurtleBot3, plus a DYNAMIXEL SDK across multiple languages. HEBI adds Ethernet-connected modular smart actuators, ROS 2 integration, camera streaming, and standardized configuration files. These are exactly the ingredients a software factory needs. ⁷

What is **not** yet practical on these platforms is a fully autonomous end-to-end self-replicator. Human assembly remains essential because physical pieces must still be sorted, aligned, wired, tightened, calibrated, and sometimes flashed or configured. Even in relatively advanced modular ecosystems, official documentation still assumes human setup steps, named devices, recommended startup poses, or physical safety checks. That makes the right near-term framing **not “robot builds robot,” but “robot-plus-software designs robot, human assembles robot, system learns what worked.”** ⁸

Platform landscape and comparison

The table below compares the main platform families against the dimensions most relevant to a human-built generational design loop. “Openness” here means **practical hackability**—documented APIs, external-compute integration, and modifiable assets—not a claim that every hardware component is open-source.

The “fit” judgment is analytical rather than vendor language. It is based on the cited hardware constraints, software interfaces, and ecosystem maturity. ⁹

Platform	Cost range from official sources	Programmability	Sensors and actuators	Mechanical modularity	Vision integration	Com and
LEGO Education SPIKE Prime	\$429.95 set, \$164.95 expansion	Blocks + Python in SPIKE app; hub runs embedded MicroPython; official support across iOS, ChromeOS, Windows, macOS, Android	6 LPF2 ports, IMU, speaker, LED matrix; color, distance, force sensors; angular motors	Strong LEGO Technic modularity; Maker Plate supports SBC attachment; Distance Sensor has LPF2 breakout access	No native camera; best paired with external webcam or Raspberry Pi Build HAT + camera	Stro class reso offic instr thiro Rasp path
LEGO MINDSTORMS Robot Inventor and EV3	Robot Inventor historical list €359.99 ; EV3 historical list \$349.99	Robot Inventor supported Scratch-style blocks + Python; Robot Inventor now programmed through SPIKE app; EV3 supports MicroPython via microSD + VS Code and MakeCode with simulator	Robot Inventor app had motors/ sensors and ML extension using device camera/mic; EV3 remains a legacy brick ecosystem	Good LEGO modularity, but both are legacy/ retired families; EV3 uses older RJ12 ecosystem	Robot Inventor ML ran on external device camera/mic; EV3 typically uses external vision	Larg histo com but supp real
LEGO Powered Up and Technic Hub	\$49.99 2-port Hub, \$89.99 Technic Hub	App-based block programming; official Powered Up programming blocks; official BLE wireless protocol for advanced external control	2-port or 4-port hubs; Technic Hub includes tilt sensor; motors, lights, selected sensors across Powered Up family	Good Technic geometry; cheaper than SPIKE; less instruction-centric than SPIKE	No onboard camera; external phone, PC, or Raspberry Pi required	Broa ecos offic prot usef cust

Platform	Cost range from official sources	Programmability	Sensors and actuators	Mechanical modularity	Vision integration	Com and
VEX V5	\$849.49 Classroom Starter Kit; \$1,689.99 Classroom Super Kit; \$384.99 Brain	Blocks, Python, C++; desktop and web workflows; web/iPad/Android use cloud compiler, Mac/Windows compile locally	21 Smart Ports + 8 3-wire ports; smart motors, inertial, optical, distance, AI Vision, GPS; 8 stored program slots	Strong metal channels, shafts, gears, pneumatics, CAD files, included Clawbot instruction manual	Native AI Vision Sensor supports color, AprilTags, and selected AI object detection	Very docs class com ecos
Modular Robotics Cubelets	\$225 Discovery Set, \$399 Curiosity Set, \$43 Bluetooth Hat, many blocks \$36–\$54	No-code, Blockly, and C in Cubelets Console; custom C API requires OS4 and Bluetooth Hat; web app in Chrome	Simple sense/think/act blocks: distance, brightness, drive, rotate, flashlight, etc.	Very fast magnetic modularity; LEGO Brick Adapters available; low mechanical DOF	No native camera block; external webcam/device required	Exce com thin educ fast com
HEBI modular actuators and kits	\$3,000 T-series actuator; arms roughly \$14,000–\$30,340 ; mobile kits roughly \$16,100–\$102,000	MATLAB, Python, C++, ROS 2; networked APIs; standardized files; high-rate control	Smart series-elastic actuators with encoders and IMU; modular arms, bases, camera modules	Very high modularity with standardized connecting hardware, Ethernet, 24–48V power, M5 mounts, daisy-chained networking	CR1 camera streams 1080p over RTSP; Scope and APIs support camera/control workflows	High rese ecos ROS exam conv URD

Platform	Cost range from official sources	Programmability	Sensors and actuators	Mechanical modularity	Vision integration	Com and
Open-source ROBOTIS ecosystem	OpenMANIPULATOR-X \$1,629.09 ;	ROS / ROS 2, MoveIt, Gazebo, DYNAMIXEL SDK in C, C++, Python, Java, MATLAB, ROS 2	DYNAMIXEL servos,	Open-source hardware/	External compute is standard; TurtleBot4 is ROS 2-based with onboard Pi, OAK-D camera, LiDAR; OpenMANIPULATOR integrates with TurtleBot3	Stro rese com ROS tool docs
	TurtleBot3 Burger \$758.20–\$783.50 ;		open manipulator,	software; STL files;		
	TurtleBot3 Waffle Pi		mobile bases;	manipulators and bases		
	\$1,642.80–\$1,933.61		TurtleBot4 adds OAK-D camera and LiDAR	combine into mobile-manipulator systems		

For a new project, the practical hierarchy is straightforward. **SPIKE Prime** is the best LEGO-native base for a first educational prototype because the hardware, sensors, app, Python path, and Raspberry Pi integration are already aligned. **VEX V5** is the best mid-range platform when you want more deterministic mechanics, stronger structures, clearer device APIs, and a better built-in vision story. **ROBOTIS** is the best open migration path once the generational loop itself has been proven. **HEBI** is the technically strongest modular robotics substrate in this list, but it is usually best treated as a later-stage upgrade rather than a first experiment because of cost. **Cubelets** are exceptional for early pedagogy and behavior composition, but they do not provide enough structural or actuation richness for a substantive multi-generation morphology program. **MINDSTORMS**, finally, is now best treated as a legacy asset, not a strategic base. ¹⁷

Representative official visuals for these platforms are already available on the cited product and manual pages for SPIKE Prime, VEX V5, Cubelets Discovery Set, HEBI robot kits, and TurtleBot 4, so the user is not dependent on community mirrors or reseller photos to understand the hardware form factors. ¹⁸

Design-synthesis and vision architecture

The crucial architectural choice is to represent morphology as a **searchable assembly grammar** rather than unrestricted geometry. The literature supports that approach strongly. Early automated physical robot design studies evolved machines from primitive building blocks; more recent work shows that differentiable or evolutionary optimization can search robot shape and controller space much faster than earlier brute-force evolutionary pipelines; and newer work such as Text2Robot explicitly adds manufacturability, electronics placement, and real-world build constraints to the optimization problem. A learning robot that proposes successor generations should therefore optimize over *typed modules*—for example, “2-wheel base + short arm + color sensor mast” or “differential drive + force-sensor bumper + gripper”—instead of trying to invent arbitrary CAD solids from scratch. ⁶

For the digital representation layer, today’s best stack is heterogeneous by purpose. **URDF** is the standard ROS description format for robot geometry and kinematic organization, and ROS’s official tutorials cover physical, collision, and inertial properties directly. **SDF** format is more expressive for simulation worlds and robot models, and Gazebo’s own tutorial on a 4-bar linkage explicitly notes that URDF’s tree structure does not natively represent closed kinematic loops while SDF format’s graph structure can. **BrickLink Studio** is the most practical LEGO-oriented instruction and bill-of-materials tool because it creates instructions, exports

`.ldr`, `.mpd`, `.csv`, `.tsv`, and `.xml`, and uses part and price information from BrickLink. **HEBI** adds a YAML robot-config plus HRDF workflow and even provides conversion help toward URDF in its ROS 2 stack.

19

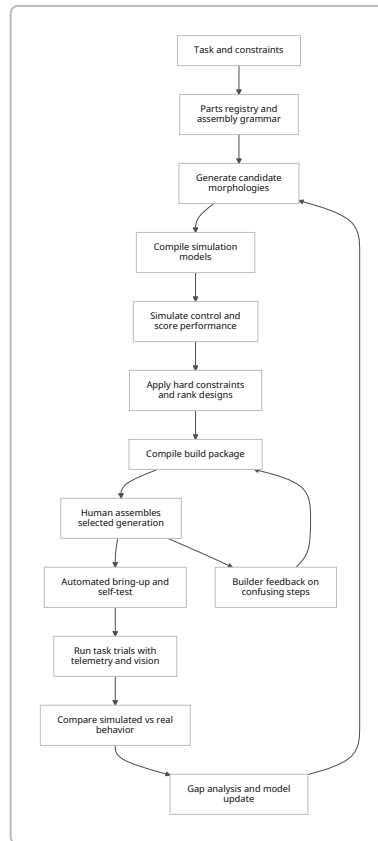
That leads to a practical layered design-synthesis toolchain. A generator emits a high-level morphology spec, a compiler lowers that into **URDF/Xacro or SDF** for simulation, a second compiler lowers it into **platform-specific build artifacts** such as Studio `.io` / LDraw files for LEGO, VEX CAD/BOM artifacts for VEX, or HEBI robot-config/HRDF for HEBI. The simulation layer then evaluates locomotion, manipulation, reachability, sensor visibility, and stability. The ranking layer applies hard constraints such as port budgets, wire lengths, motor limits, closed-loop compatibility, sensor mounting clearances, and human assembly complexity. Only the surviving design candidates become buildable generations. 20

Vision integration is the dividing line between “toy programmability” and an actual learning factory. The educational hubs themselves rarely have enough onboard compute or camera capability to run the full learning and design loop. On LEGO, the practical path is **external vision**: Robot Inventor’s machine-learning extension used the camera or microphone of the connected device, SPIKE Prime officially integrates with Raspberry Pi via the Maker Plate and Build HAT, and the Build HAT docs explicitly note that LEGO Technic devices can be controlled in Python alongside a Raspberry Pi camera module. VEX is the strongest “middle” option because it has a native **AI Vision Sensor** supporting color, AprilTags, and selected AI object classes. HEBI’s **CR1** module streams 1080p RTSP video over the network. TurtleBot 4 already ships with onboard Raspberry Pi compute, OAK-D camera, LiDAR, and ROS 2. Cubelets has no native camera block, so vision necessarily lives offboard. 21

The best design constraint set for a first implementation is narrower than many teams initially imagine. It should include at least: allowed structural parts; allowed sensor and actuator classes; port and power budgets; stable support polygon or center-of-mass checks; motion interference checks; reachable and visible sensor fields; preferred assembly symmetries; and a hard cap on **human build complexity**, such as maximum number of unique fastener types or maximum cable crossings. This is partly an inference, but it follows directly from the official hardware limits of the listed platforms and from the simulation-representation constraints documented by ROS and Gazebo. 22

Human-in-the-loop build and verification workflow

A successful human-in-the-loop generational system should treat the human not as an ad hoc assembler, but as a **formal manufacturing station**. The software factory should emit a design package containing a bill of materials, ordered build steps, exploded or staged views, a cable and port map, an initialization program, and a short acceptance-test script. On LEGO-oriented workflows, BrickLink Studio already supports instructions, parts lists, and export formats. VEX kits ship with starter build documentation and CAD resources. HEBI’s configuration files encode expected module names, gains, and kinematic structure in machine-readable form. These existing tools are enough to make the human build step reproducible. 23



The loop above is rigorous enough to behave like a factory, not just a build-and-hope cycle. The key step is **I**, automated bring-up and self-test. Many of the candidate platforms already support the ingredients for this. SPIKE auto-detects motors and sensors on LPF2 ports, and its hub supports stored programs navigated from the device itself. VEX smart ports auto-detect devices and the Brain exposes diagnostics. HEBI objects are discovered on the network and organized by family/name conventions. Cubelets Console and Bluetooth Hat provide device-level programming and data logging. A factory should exploit those capabilities by running a short acceptance routine: identify devices, wiggle each actuator, read each sensor once, confirm expected port identities, and then compare the detected assembly against the design specification. ²⁴

Error tolerance is not a side issue; it is central. A generational builder-friendly system should prefer **symmetrical subassemblies**, use part-color conventions where possible, minimize ambiguous mirrored steps, and add verification checkpoints after every structurally important stage. A LEGO system can exploit Studio-style staged instructions and part counts. VEX can exploit rigid port naming and metal-assembly repeatability. HEBI and ROBOTIS can exploit named-chain conventions and kinematic descriptors. The result is a build process that can tolerate human error without silently poisoning the learning loop. ²⁵

The “observation, gap analysis, planning” loop should be explicit. After every physical build, the system should compare simulation predictions against real telemetry and video. If a design failed because of structural flex, loose supports, motor saturation, wheel slip, or sensor occlusion, that failure should update not only the controller model but also the **design prior** and the **builder-error model**. That two-level learning—controller learning within a generation, design learning across generations—is the closest practical analogue to the “learning robot that designs successive generations” idea in the current state of the art. ²⁶

Prototype roadmap, software stack, and data formats

A good first roadmap is staged, because the main risk is not whether one robot can be programmed, but whether the **whole loop** can be made reliable. Given that budget, timeline, and user skill are unspecified, the most defensible roadmap is a three-stage progression: an **education-first low-cost prototype**, an **engineering-stable second prototype**, and an **open research migration path**. That sequencing follows directly from the maturity and affordances documented for SPIKE Prime, VEX V5, and the ROBOTIS / TurtleBot ecosystems. ²⁷

Stage one should prove the loop, not the ultimate robot. Use **SPIKE Prime plus external compute** or **SPIKE Prime plus Raspberry Pi Build HAT**. Limit the morphology grammar to a small family of mobile robots: different wheelbases, wheel diameters, sensor mast heights, bumper geometries, and lightweight end-effectors. Build only one selected design per generation. Success at this stage means the system can generate a candidate, compile instructions, let a human build it, run it, and use the results to change the next design. SPIKE is ideal here because the set includes motors, sensors, a programmable hub, official learning resources, and an official SBC integration path. ²⁸

Stage two should harden mechanics and perception. Move the same generational loop to **VEX V5** if you want stronger structures, more deterministic actuation, and native smart-device introspection. This is the stage where a software factory starts to feel industrial rather than educational: stiffer frames, richer device APIs, AprilTag-capable onboard vision, clearer diagnostics, and more reproducible assembly. The VEX Brain's port model, stored program slots, and web/desktop workflows make commissioning and reruns much more disciplined than with a pure tablet-centric kit. ²⁹

Stage three should separate the learning architecture from any one kit vendor. At that point, migrate the design representation and evaluation stack to **URDF/Xacro + SDF + ROS 2 + Gazebo**, and target **ROBOTIS OpenMANIPULATOR-X / TurtleBot3** for research-oriented openness. If the project later needs higher payload, finer dynamics, or richer sensors, migrate again to HEBI. This staged migration keeps the software factory stable while allowing the hardware substrate to evolve. It also reduces the risk of long-term dependence on a retired or narrowly educational ecosystem. ³⁰

The recommended software stack is therefore layered. Use a **design registry** for inventory and constraints; **Python** as the orchestration language; **ROS 2** for graph integration where appropriate; **URDF/Xacro** and **SDFFormat** for simulation models; **Gazebo** for fast design iteration; **BrickLink Studio** if the platform is LEGO and you want machine-generated human instructions; **OpenCV and fiducials** or platform-native camera APIs for vision; and a **test harness** that runs identical acceptance routines after every assembly. If using Build HAT, note that Raspberry Pi's current documentation says it is not yet supported on Raspberry Pi OS Trixie and recommends Bookworm for now. ³¹

The most useful output formats for the human-build step are:

- **LEGO-oriented:** Studio `.io`, LDraw `.ldr/.mpd`, and exported `.csv/.xml` BOM files, because Studio already supports instruction making and multiple exports. ³²
- **ROS-oriented:** `URDF` / `Xacro` for kinematics and `SDF` for simulation-rich models or closed-loop mechanisms. ³³
- **HEBI-oriented:** YAML robot-config plus HRDF and gains files. ³⁴

A practical example of a high-level design manifest could look like this:

```
{
  "generation_id": "g012",
  "platform": "spike_prime",
  "task": "line_follow_and_push",
  "constraints": {
    "max_ports": 6,
    "max_unique_part_types": 25,
    "must_include": ["color_sensor", "distance_sensor"],
    "forbid_closed_loops": true,
    "max_build_steps": 40
  },
  "morphology": {
    "chassis": "diff_drive_short",
    "wheel_diameter_mm": 56,
    "sensor_mast": "front_low",
    "bumper": "springless",
    "tool": "passive_plow"
  },
  "electronics": {
    "hub_ports": {
      "A": "left_motor",
      "B": "right_motor",
      "C": "color_sensor",
      "D": "distance_sensor"
    }
  },
  "controller_targets": {
    "sim_policy": "ppo_v3",
    "download_artifact": "g012_controller.py"
  },
  "acceptance_tests": [
    "port_map_check",
    "motor_direction_check",
    "sensor_smoke_test",
    "drive_1m_trial"
  ]
}
```

A corresponding build package for the human station could be this:

```
generation_id: g012
bom:
  - part: "spike_large_hub"
```

```

    qty: 1
  - part: "medium_angular_motor"
    qty: 2
  - part: "color_sensor"
    qty: 1
  - part: "distance_sensor"
    qty: 1
  - part: "technic_beam_11"
    qty: 4
  - part: "wheel_56mm"
    qty: 2

assembly_steps:
  - step: 1
    action: "build left chassis rail"
    verify: "photo_required"
  - step: 2
    action: "mirror for right chassis rail"
    verify: "length_match"
  - step: 3
    action: "mount hub and motors"
    verify: "free_spin"
  - step: 4
    action: "connect ports A/B/C/D"
    verify: "port_map_check"
  - step: 5
    action: "mount sensors and bumper"
    verify: "sensor_visibility_check"

commissioning:
  firmware_artifact: "g012_controller.py"
  tests:
    - "hub_identify"
    - "motor_wiggle"
    - "sensor_read"
    - "straight_drive_2s"

```

The most important documentation anchors for implementation are the official **SPIKE Prime Python and technical-specification pages**, the official **LEGO BLE Wireless Protocol documentation**, the official **VEX V5 API reference**, the official **Cubelets C API**, the official **HEBI documentation and robot-config format**, the official **OpenMANIPULATOR-X and DYNAMIXEL SDK docs**, the official **ROS 2 URDF tutorials**, the official **Gazebo and SDF format docs**, and the official **BrickLink Studio instruction/export docs**. Those documents are enough to bootstrap the whole stack without depending on secondary blog posts. 35

Risks, safety, and recommended decisions

The main technical risk is **simulation-to-real mismatch**. This is not speculative; it is a known problem across automated robot-design research, and it is the reason the best design papers either test in reality or explicitly reason about manufacturability and transfer. The mitigation is to start with narrow tasks, stiff or predictable morphologies, simple terrain, and aggressive acceptance testing, then learn a residual model from the mismatch between simulation and real telemetry. That recommendation follows directly from both the classic and modern automated-design literature. ³⁶

The main workflow risk is **instruction failure**. If a human cannot reliably assemble the design package, the data generated by the built robot are not trustworthy. The mitigation is to treat documentation as a compiler artifact: every generation gets a BOM, ordered steps, expected port map, current draw or motion sanity checks, and a builder feedback channel that updates the instruction generator. BrickLink Studio's instruction-oriented workflow, VEX's structured kits and CAD resources, and HEBI's machine-readable configuration files make this realistic. ³⁷

The main ecosystem risk is **platform retirement or lock-in**. LEGO has already retired Robot Inventor, and EV3 is legacy. That does not make those kits useless, but it makes them poor foundations for a long-horizon software factory. The mitigation is to keep the core design representation and learning loop platform-agnostic, prove the concept on SPIKE or VEX, and then migrate the same factory to ROS-native hardware such as ROBOTIS or HEBI if the project continues. ³⁸

There are also ordinary hardware safety issues that become more serious as the project scales. VEX pneumatics run at 100 psi, VEX distance sensing uses a Class 1 laser, the Raspberry Pi Build HAT requires an external 8V supply for motors and some sensors, HEBI actuators run on 18-48V and can become hot under load, and both OpenMANIPULATOR and TurtleBot manipulation docs include explicit warnings about joint pose, physical damage, and pinch danger. A generational factory must therefore include a safety envelope: speed limits during commissioning, current/temperature limits where available, guarded startup poses, and a human approval gate before full-power tests. ³⁹

The final decision guidance is clear. If the aim is to **teach the concept and get a first closed-loop prototype running quickly**, choose **SPIKE Prime plus external compute**. If the aim is to **build a serious mechanical evolution platform without immediately jumping to ROS-heavy research infrastructure**, choose **VEX V5**. If the aim is to **build a lasting, open, research-grade generational design system**, choose **ROBOTIS OpenMANIPULATOR-X and TurtleBot3/4** once the first loop works. If the aim is to **push modular actuation, sensing, and research fidelity as far as possible**, choose **HEBI** and budget accordingly. Do **not** start a new long-term program on retired MINDSTORMS unless the legacy hardware is already in hand and the short-term goal is purely educational. ⁴⁰

The bottom-line feasibility judgment is therefore: **highly feasible as a bounded human-built software factory; weakly feasible as an open-ended self-designing self-fabricating robot**. The right first implementation is not a monolithic "learning robot," but a **design-service + simulation-service + instruction-service + human build station + robot test cell**. With that framing, the concept is not only feasible; it is a sensible and rigorous robotics program. ⁴¹

1 Get the LEGO® Education SPIKE™ App

https://education.lego.com/en-us/teacher-resources/lego-education-spike-prime/support-technical-info/lego-education-spike-prime-support-technical-info-get-the-lego-education-spike-app/?utm_source=chatgpt.com

2 6 36 <https://www.nature.com/articles/35023115>

<https://www.nature.com/articles/35023115>

3 7 22 24 3030187_Techsspecs_TechnicLargeHub_v2.indd

https://assets.education.lego.com/v3/assets/blt293eea581807678a/bltf512a371e82f6420/5f8801baf4f4cf0fa39d2feb/techspecs_techniclargehub.pdf?locale=en-us

4 10 17 18 27 28 40 <https://education.lego.com/en-us/products/lego-education-spike-prime-set/45678>

<https://education.lego.com/en-us/products/lego-education-spike-prime-set/45678>

5 19 20 30 31 33 <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>

8 Help Topics

https://www.lego.com/en-us/service/help-topics/article/about-the-lego-mindstorms-robot-inventor-app?utm_source=chatgpt.com

9 <https://lego.github.io/lego-ble-wireless-protocol-docs/>

<https://lego.github.io/lego-ble-wireless-protocol-docs/>

11 38 Help Topics

https://www.lego.com/en-us/service/help-topics/article/About-MINDSTORMS-Robot-Inventor?utm_source=chatgpt.com

12 <https://www.lego.com/en-us/product/hub-88009>

<https://www.lego.com/en-us/product/hub-88009>

13 VEX V5 Classroom Starter Kit - VEX Robotics

https://www.vexrobotics.com/v5-classroom-starter-kit.html?utm_source=chatgpt.com

14 <https://modrobotics.com/cubelets-for-education/>

<https://modrobotics.com/cubelets-for-education/>

15 <https://shophebi.com/products/t-series-actuators>

<https://shophebi.com/products/t-series-actuators>

16 OpenMANIPULATOR-X

https://emanual.robotis.com/docs/en/platform/openmanipulator_x/overview/?utm_source=chatgpt.com

21 LEGO® MINDSTORMS® app | Official LEGO® Shop US

https://www.lego.com/en-us/mindstorms/apps/ev3-programmer-app?utm_source=chatgpt.com

23 32 37 <https://studiohelp.bricklink.com/hc/en-us/articles/5404381697559-Introduction-to-Studio>

<https://studiohelp.bricklink.com/hc/en-us/articles/5404381697559-Introduction-to-Studio>

25 <https://studiohelp.bricklink.com/hc/en-us/articles/5627862550935-Parts-List-Settings>

<https://studiohelp.bricklink.com/hc/en-us/articles/5627862550935-Parts-List-Settings>

26 <https://arxiv.org/abs/2102.02202>

<https://arxiv.org/abs/2102.02202>

29 Understanding the V5 Robot Brain – VEX Library

https://kb.vex.com/hc/en-us/articles/360060662352-Understanding-the-V5-Robot-Brain?utm_source=chatgpt.com

34 HEBI Robotics | Documentation

https://docs.hebi.us/tools.html?utm_source=chatgpt.com

35 <https://education.lego.com/en-us/product-resources/spike-prime/downloads/python/>

<https://education.lego.com/en-us/product-resources/spike-prime/downloads/python/>

39 V5 Pneumatics Kit - VEX Robotics

https://www.vexrobotics.com/276-8750.html?utm_source=chatgpt.com

41 <https://pmc.ncbi.nlm.nih.gov/articles/PMC10576117/>

<https://pmc.ncbi.nlm.nih.gov/articles/PMC10576117/>